



Today's agenda

↳ DSU

↳ Problems on DSU



AlgoPrep

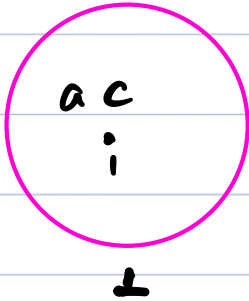
DSU



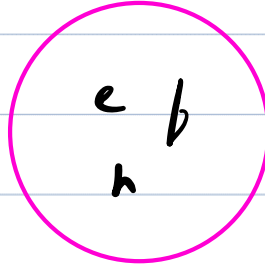
↳ Disjoint Set Union → creating sets by following Property?

↓
Given in the
Problem
Statement

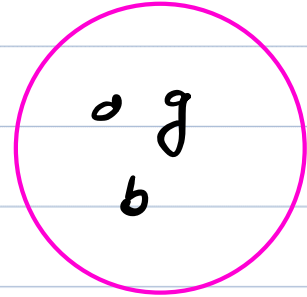
a b c d e f g h i



1



2



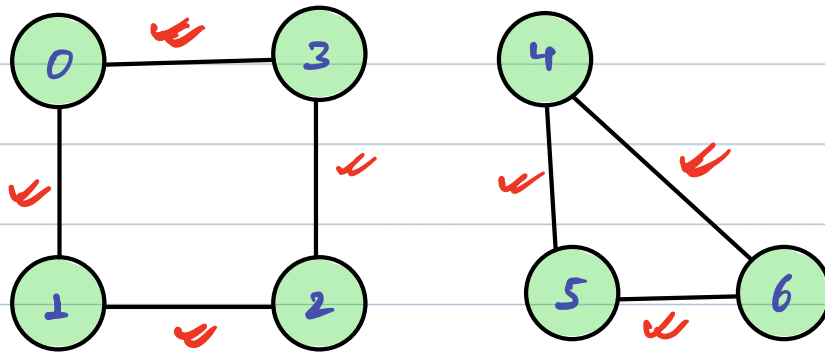
3



AlgoPrep



Q) No. of Connected Component in a graph.

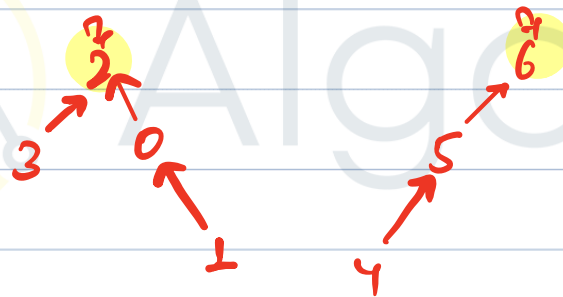


edges:



Par:

0	1	2	3	4	5	6
2	2	2	2	2	2	2



x	y	P(x)	P(y)
1	2	0	2
2	3	2	3
0	3	2	2
4	5	4	5
5	6	5	6
4	6	6	6



// Pseudo code → v1

```
int Par[N];
```

```
int ConnectedComponents (int [N][2] edges, int N) {
```

```
    for (int i=0; i<N; i++) {
```

```
        Par[i] = i;
```

```
    }
```

```
    for (int i=0; i<m; i++) {
```

```
        int x = edges[i][0];
```

```
        int y = edges[i][1];
```

```
        union(x, y);
```

```
    }
```

```
    int ans = 0;
```

```
    for (int i=0; i<N; i++) { if (Par[i] == i) { ans++; } }
```

```
    return ans;
```

```
}
```

$O(E \times V)$ ←

```
Public void union (int x, int y) {
```

```
    int Px = find(x);
```

```
    int Py = find(y);
```

```
    if (Px == Py) { return; }
```

```
    Par[Px] = Py;
```

```
}
```

T.C: $O(V)$ ←



T.C: $O(V)$ ←

```

Public int find (int x) {
    if (x == Par[x]) { return x; }

    int temp = find (Par[x]);
    return temp;
}

```

Tracing

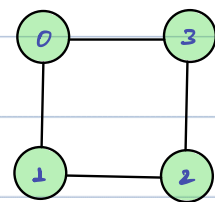
```

int Par[N];
int ConnectedComponents (int m, int edges, int N) {
    for (int i = 0; i < N; i++) {
        Par[i] = i;
    }

    for (int i = 0; i < m; i++) {
        int x = edges[i][0];
        int y = edges[i][1];

        union(x, y);
    }
}

```

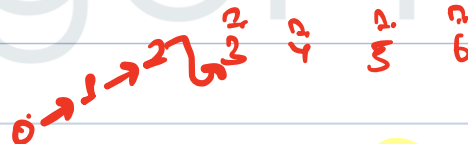


edges:

0	1
1	2
2	3
0	3
4	5
5	6
4	6

Par:

0	1	2	3	4	5	6
1	2	3	3	4	5	6



x	y	px	py
0	1	0	1
1	2	1	2
2	3	2	3
0	3	3	3

```

Public void union (int x, int y) {
    int px = find (x);
    int py = find (y);

    if (px == py) { return; }
    Par[px] = py;
}

```

```

Public int find (int x) {
    if (x == Par[x]) { return x; }

    int temp = find (Par[x]);
    return temp;
}

```

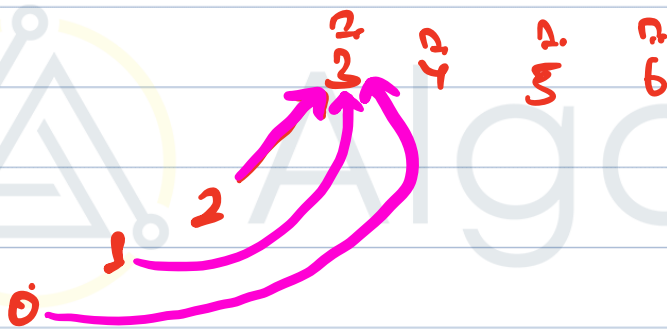
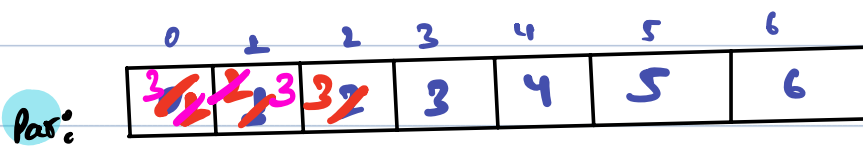


* Optimization of DSU

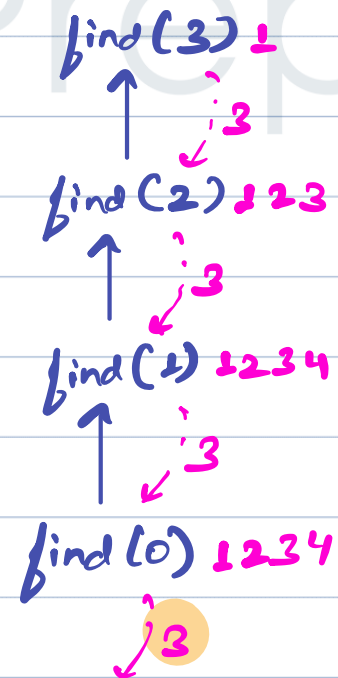
↳ (i) Path compression.

(ii) Union by rank.

(i) Path Compression.



```
Public int find (int n) {  
    1 if (n == Par[n]) { return n; }  
    2 int temp = find (Par[n]);  
    3 Par[n] = temp;  
    4 return temp;  
}
```

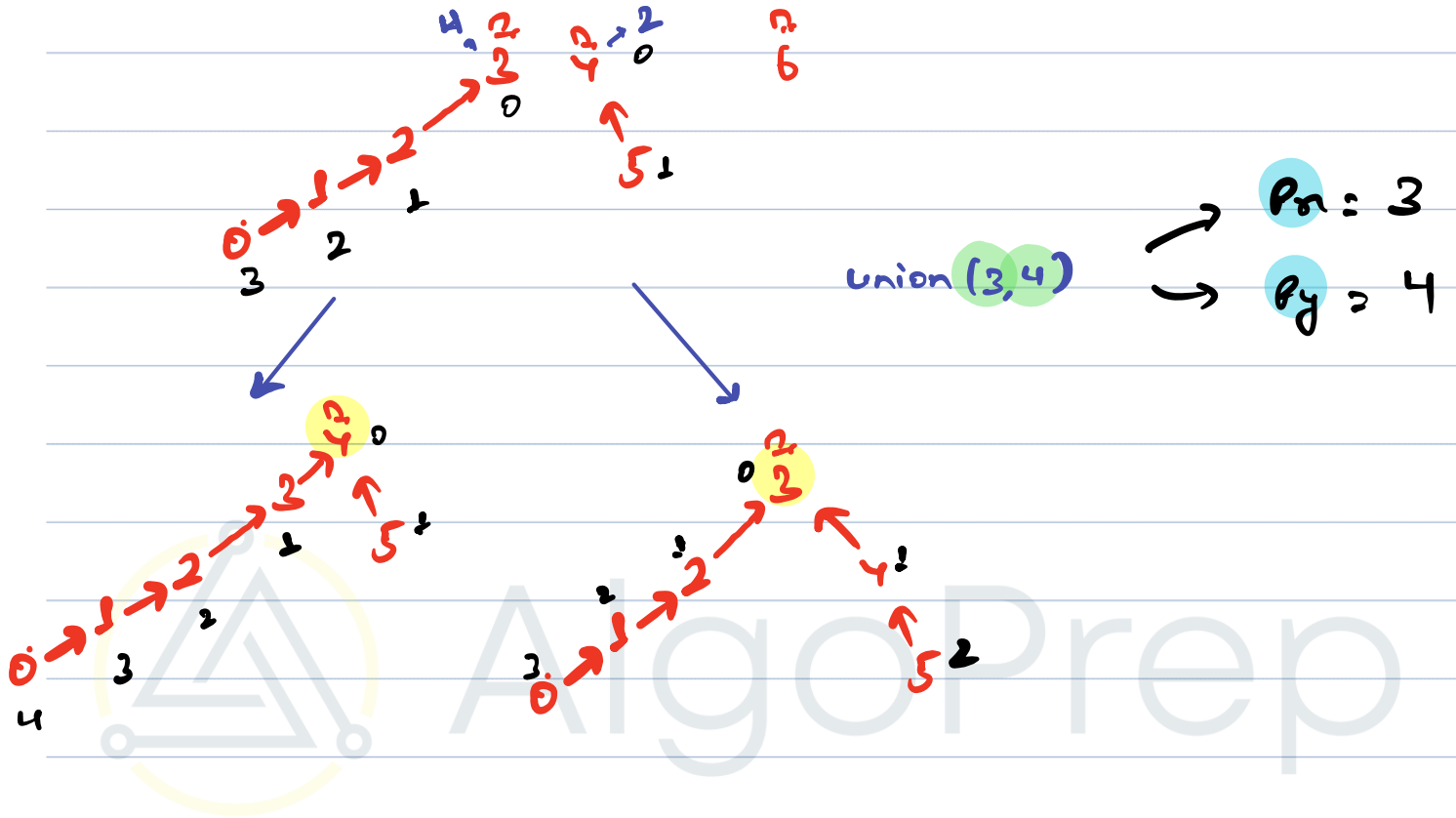




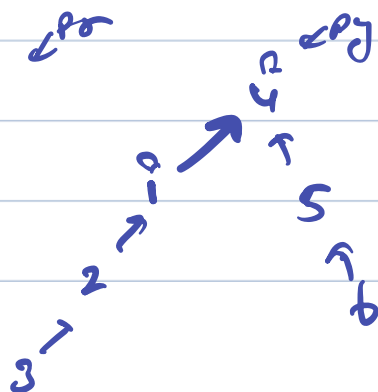
⑪ Union by rank

par:

0	1	2	3	4	5	6
2	1	3	3	4	4	6



↳ while doing union, you make leader of longer chain as your new leader.





// Pseudo code $\rightsquigarrow$ v2

int Par[N];

int rank[N];

int ConnectedComponents (int [N][2] edges, int N) {

for (int i=0; i<N; i++) {

Par[i] = i;

rank[i] = 1;

}

for (int i=0; i<m; i++) {

int x = edges[i][0];

int y = edges[i][1];

union (x, y);

}

int ans = 0;

for (int i=0; i<N; i++) { if (Par[i] == i) { ans++; } }

return ans;

}

Public void union (int x, int y) {

int Px = find(x);

int Py = find(y);

if (Px == Py) { return; }

if (rank[Px] > rank[Py]) {

Par[Py] = Px;

} else if (rank[Px] < rank[Py]) {

Par[Px] = Py;

} else { Par[Px] = Py; rank[Py]++; }

}

union by rank

$O(V)$

$O(E)$

r.c: $O(V+E)$

$O(1)$



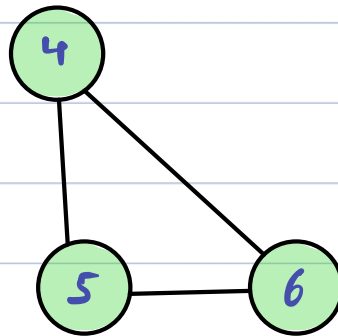
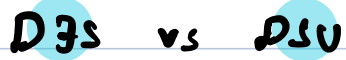
```
Public int find (int n) {  
    if (n == par[n]) { return n; }  
    int temp = find (par[n]);  
    par[n] = temp; ] Path Compression  
    return temp;  
}
```

$O(1)$ ←

→ Amortized analysis : A big data set of cases and then we come up with time complexity.



AlgoPrep



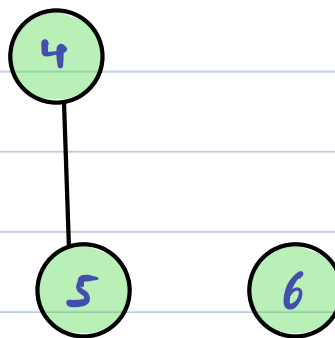
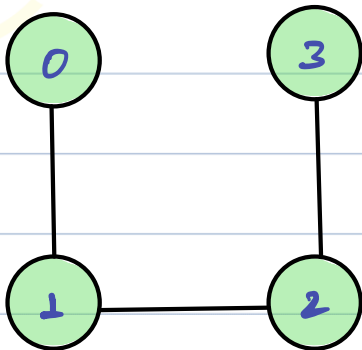
075

$O(V+E)$
(better)

DSU

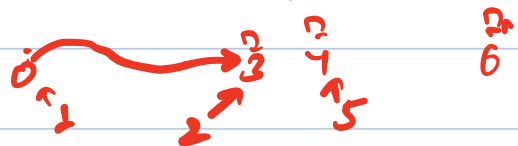
$$O(V + E)$$

Dynamic graph



035

DSV

$$O(E * (v + E))$$
 $O(E + V)$ 

ans = 7 6 5 4 3



Q) Remove max number of edges to keep graph fully traversable.

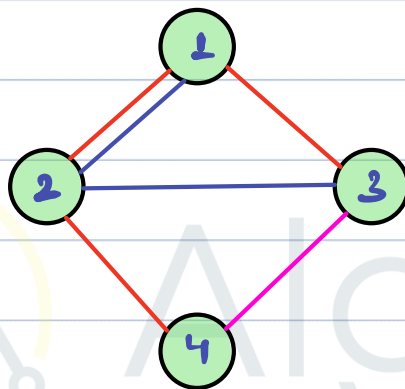
→ A graph with 3 types of edges are given:

Type 1: Can be traversed by Alice only. —

Type 2: Can be traversed by Bob only. —

Type 3: Can be traversed by both Alice and Bob. —

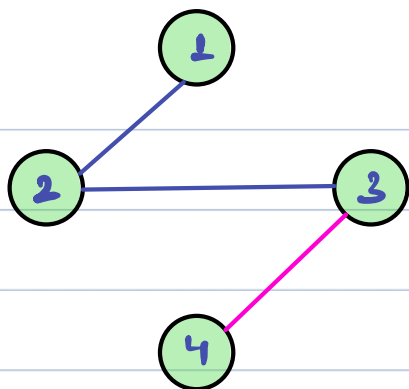
Ex:



	0	1	2
0	3	1	2
1	3	2	3
2	1	1	3
3	1	2	4
4	1	1	2
5	2	2	4

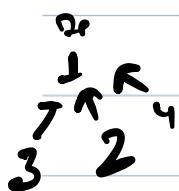
Idea

↳ remove max edge == keep min no. of edge.

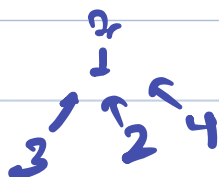


	0	1	2	
0	3	1	2	✓
1	3	2	3	✓
2	1	1	3	✓x ✓
3	1	2	4	✓
4	1	1	2	✓x ✓
5	2	2	4	✓

Alice



Bob



low = 2

//PSuedo Code

```

Public boolean union (int x, int y, int[] Parent, int[] Rank) {
    int Pn = find(x);
    int Py = find(y);
    if (Pn == Py) { return false; }
    if (Rank[Pn] > Rank[Py]) {
        Par[Py] = Pn;
    }
    else if (Rank[Pn] < Rank[Py]) {
        Par[Pn] = Py;
    }
    else {
        Par[Pn] = Py;
        Rank[Py]++;
    }
    return true;
}

```



```
int maxEdgeRemoval (int edges[M][3], int N) {  
    Arrays.sort ( edges, (a,b) -> Integer.compare (b[0], a[0]) );
```

```
    int Parenta[N+1];
```

```
    int ranka[N+1];
```

```
    int Parentb[N+1];
```

```
    int rankb[N+1];
```

```
    for (int i=1; i<=N; i++) {
```

```
        Parenta[i] = Parentb[i] = i;
```

```
        ranka[i] = rankb[i] = 1;
```

```
    }
```

```
    int counta = 1;  
    int countb = 1;
```

```
    int ans = 0;
```

```
    for (int i=0; i<edges.length; i++) {
```

```
        int cat = edges[i][0];
```

```
        int x = edges[i][1];
```

```
        int y = edges[i][2];
```

```
        if (cat == 3) {
```

```
            bool mergea = union (x, y, Parenta, ranka);
```

```
            bool mergeb = union (x, y, Parentb, rankb);
```

```
            if (mergea == false && mergeb == false) {
```

```
                ans++;
```

```
            }
```

```
            if (mergea == true) { counta++; }
```

```
            if (mergeb == true) { countb++; }
```

```
        }
```

```
else if (cat == 2) {
```

```
    bool mergeb = union(x, y, Parentb, rankb);
```

```
    if (mergeb == false) {
```

```
        ans++;
```

```
    } else { countb++; }
```

```
}
```

```
else {
```

```
    bool mergea = union(x, y, Parenta, ranka);
```

```
    if (mergea == false) {
```

```
        ans++;
```

```
    } else { counta++; }
```

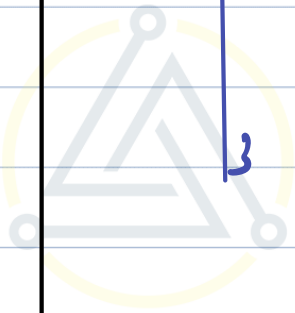
```
}
```

```
}
```

```
if (counta != n || countb != n) { return -1; }
```

```
return ans;
```

```
}
```



AlgoPrep